zeroriscy_prefetch_buffer and zeroriscy_fetch_fifo

OVERVIEW

The instruction fetch stage is the front-end of the ZeroRISCV processor and is responsible for continuously supplying valid instructions to the decode stage. It ensures smooth instruction flow by handling instruction memory access, branch redirection, memory latency, and support for compressed RISC-V (RVC) instructions.
This fetch system is mainly built using two important RTL modules:

- zeroriscy_prefetch_buffer
- zeroriscy_fetch_fifo

Together, these modules form a producer-consumer pipeline where the prefetch buffer fetches instructions from memory and the fetch FIFO aligns, stores, and forwards them to the decode stage.
The design ensures continuous instruction supply, proper handling of memory delays, clean flushing during branches, support for 16-bit compressed instructions, and back-pressure handling from the decode stage.

MODULE 1: zeroriscy_prefetch_buffer - PURPOSE

The Prefetch Buffer acts as the instruction-memory side master of the fetch stage.
Its primary responsibilities are:

- Sending instruction fetch requests to memory
- Handling request/grant/valid protocol
- Managing fetch addresses
- Supporting branch redirection
- Absorbing instruction memory latency
- Feeding fetched instructions into the downstream FIFO

It ensures that instruction fetching continues efficiently even when memory responses take multiple cycles.

MAIN WORKING PRINCIPLE

The prefetch buffer communicates with instruction memory using:

- instr_req_o -> Request signal
- instr_gnt_i -> Grant signal
- instr_rvalid_i -> Valid returned data signal

It uses a Finite State Machine (FSM) to manage fetch operations and handle exceptional cases such as branches during pending fetches.

FSM STATES

The module uses four major states:

- IDLE : No active memory request
- WAIT_GNT : Waiting for memory grant
- WAIT_RVALID : Waiting for instruction data
- WAIT_ABORTED : Waiting to discard stale instruction after branch

This FSM ensures safe and correct instruction fetching under all conditions.

STATE DESCRIPTION: IDLE & WAIT_GNT

1. IDLE

In this state, no outstanding memory request exists, and new fetch requests can be issued.
If req_i = 1 and the FIFO has free space (fifo_ready = 1), then a new request is sent.
If memory grants immediately -> Move to WAIT_RVALID. Otherwise -> Move to WAIT_GNT.

2. WAIT_GNT

In this state, the request remains active, and the buffer waits for a memory grant.
Once instr_gnt_i = 1, the FSM moves to WAIT_RVALID.

STATE DESCRIPTION: WAIT_RVALID & WAIT_ABORTED

3. WAIT_RVALID

In this state, memory has accepted the address, and the buffer waits for actual instruction data.
Once instr_rvalid_i = 1, the fetched instruction is pushed into the FIFO. If another request is pending, back-to-back pipelining is supported. This improves fetch throughput significantly.

4. WAIT_ABORTED

This state handles a branch during an in-flight fetch. If a branch occurs before memory returns data:

- Old instruction becomes invalid
- FIFO is cleared
- Returned stale instruction is discarded

Only after discarding stale data does the buffer start fetching from the new branch target.

ADDRESS COMPUTATION & BUSY SIGNAL

Address Computation:

The next sequential fetch address is calculated as:
fetch_addr = {instr_addr_q[31:2], 2'b00} + 4
This ensures word alignment is maintained and every fetch moves to the next 32-bit instruction word. Before adding 4, bits [1:0] are cleared to guarantee alignment.

Busy Signal:

The module provides busy_o, which indicates whether the fetch unit is still active.
Logic: busy_o = (CS != IDLE) | instr_req_o
This helps the processor determine whether the fetch pipeline is fully idle. Useful during debug halt, wait-for-interrupt (WFI), and pipeline flush conditions.

MODULE 2: zeroriscy_fetch_fifo - PURPOSE

The Fetch FIFO is a 3-entry instruction buffer placed between instruction memory and the decode stage.
Its responsibilities include:

- Buffering fetched instructions
- Decoupling memory timing from decode stage
- Supporting compressed instruction alignment

- Managing aligned and unaligned instruction delivery

This is especially important because compressed RISC-V instructions may start at half-word boundaries and can span across two 32-bit words.

INTERNAL STORAGE

The FIFO contains three parallel arrays:

- addr_Q[0:2] : Stores instruction addresses
- rdata_Q[0:2] : Stores instruction words
- valid_Q[0:2] : Stores valid bits

This allows simultaneous tracking of Data, Program Counter (PC), and the validity of each entry.

COMPRESSED INSTRUCTION HANDLING

RISC-V compressed instructions are 16-bit instructions. Their opcode condition is: opcode[1:0] != 2'b11.

This means:

- Standard instructions -> 32-bit
- Compressed instructions -> 16-bit

Compressed instructions may begin at an aligned address (addr[1] = 0) or an unaligned address (addr[1] = 1). The FIFO must correctly assemble instructions in both cases.

ALIGNMENT LOGIC

Aligned Instruction:

If addr[1] = 0, the instruction starts normally inside the current word. No extra reconstruction is required.

Unaligned Instruction:

If addr[1] = 1, the instruction starts in the upper half of one word and may continue into the next word. The FIFO combines the upper 16 bits of the current word and the lower 16 bits of the next word to reconstruct the full instruction. This makes compressed instruction execution completely transparent to the decode stage.

OUTPUT SELECTION & READY LOGIC

Output Selection Logic:

The module decides whether to use the aligned path or unaligned path based on out_addr_o[1].

Selection Rule:

- If unaligned -> Use reconstructed data from adjacent FIFO entries
- If aligned -> Use normal head word directly

This ensures correct instruction delivery for all instruction types.

Ready Logic:

The FIFO can accept new data when: in_ready_o = ~valid_Q[1].

This means as long as slot 1 is free, new instruction words can be pushed safely. Slot 2 acts as a spill slot for safety. This conservative design prevents overflow during simultaneous push and pop operations.

PUSH, POP & RESET LOGIC

Push Logic:
When in_valid_i = 1, the incoming word is written into the first available empty slot. Priority: slot0 -> slot1 -> slot2. This maintains proper FIFO order.

Pop / Advance Logic:
When out_ready_i && out_valid_o, the decode stage successfully consumes an instruction. The FIFO then advances depending on instruction alignment and size (16-bit or 32-bit).

Reset and Clear:
- Reset (rst_n = 0): All values are cleared (Valid bits -> 0, Addresses -> 0, Data -> 0).
- Clear (clear_i = 1): Used during branch flush. Only valid_Q is cleared. Stored data remains physically present but is treated as invalid. This avoids unnecessary rewriting and improves efficiency.

INTER-MODULE CONNECTION

The prefetch buffer and fetch FIFO work together using direct signal mapping:
- instr_rdata_i -> in_rdata_i : Instruction word
- fifo_valid -> in_valid_i : Push control
- fifo_ready -> in_ready_o : FIFO space check
- valid_o -> out_valid_o : Decode valid
- ready_i -> out_ready_i : Decode ready
- rdata_o -> out_rdata_o : Final instruction
- addr_o -> out_addr_o : Program Counter
- branch_i -> clear_i : Branch flush

This creates a clean producer-consumer architecture.

KEY DESIGN ADVANTAGES & SUMMARY

Key Design Advantages:
Hides instruction memory latency, supports compressed RISC-V instructions, ensures clean branch flushing and safe handling of stale fetched data, maintains high throughput through back-to-back pipelining, and provides reliable alignment of unaligned instructions.

SUMMARY:
The zeroriscy_prefetch_buffer and zeroriscy_fetch_fifo together form the complete instruction fetch front-end of the ZeroRISCV processor. The Prefetch Buffer handles memory communication, address generation, FSM-based fetch control, and branch redirection. The Fetch FIFO handles instruction buffering, compressed instruction alignment, and decode-stage delivery. Together, they ensure efficient, reliable, and high-performance instruction supply to the processor pipeline.